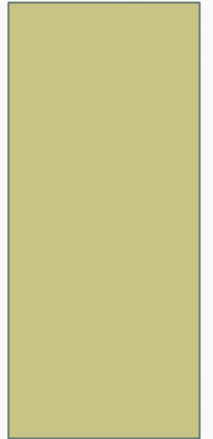


EC-200 DATA STRUCTURES

LECTURE 12- AVL TREE

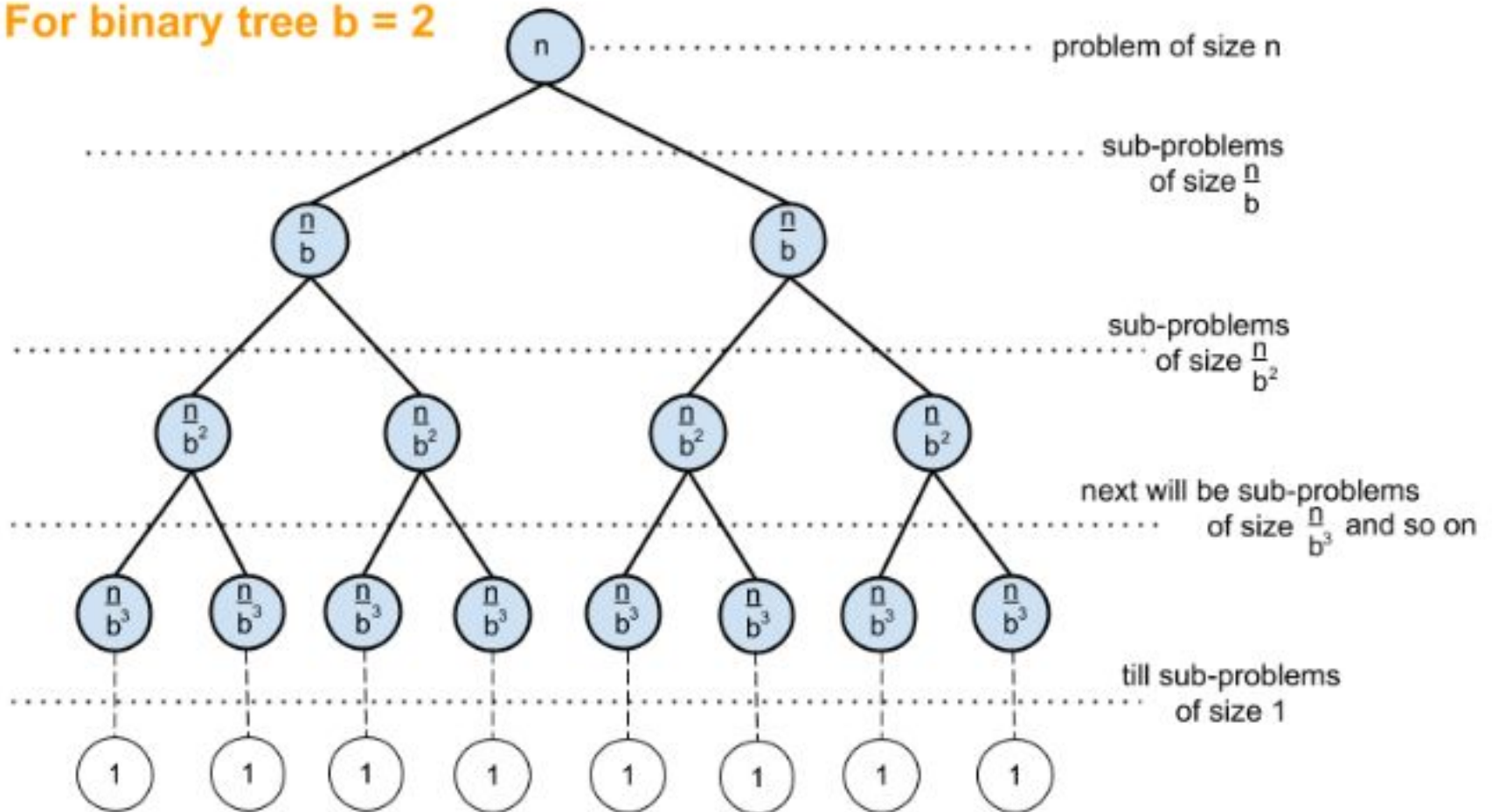


BINARY SEARCH TREE - BEST TIME

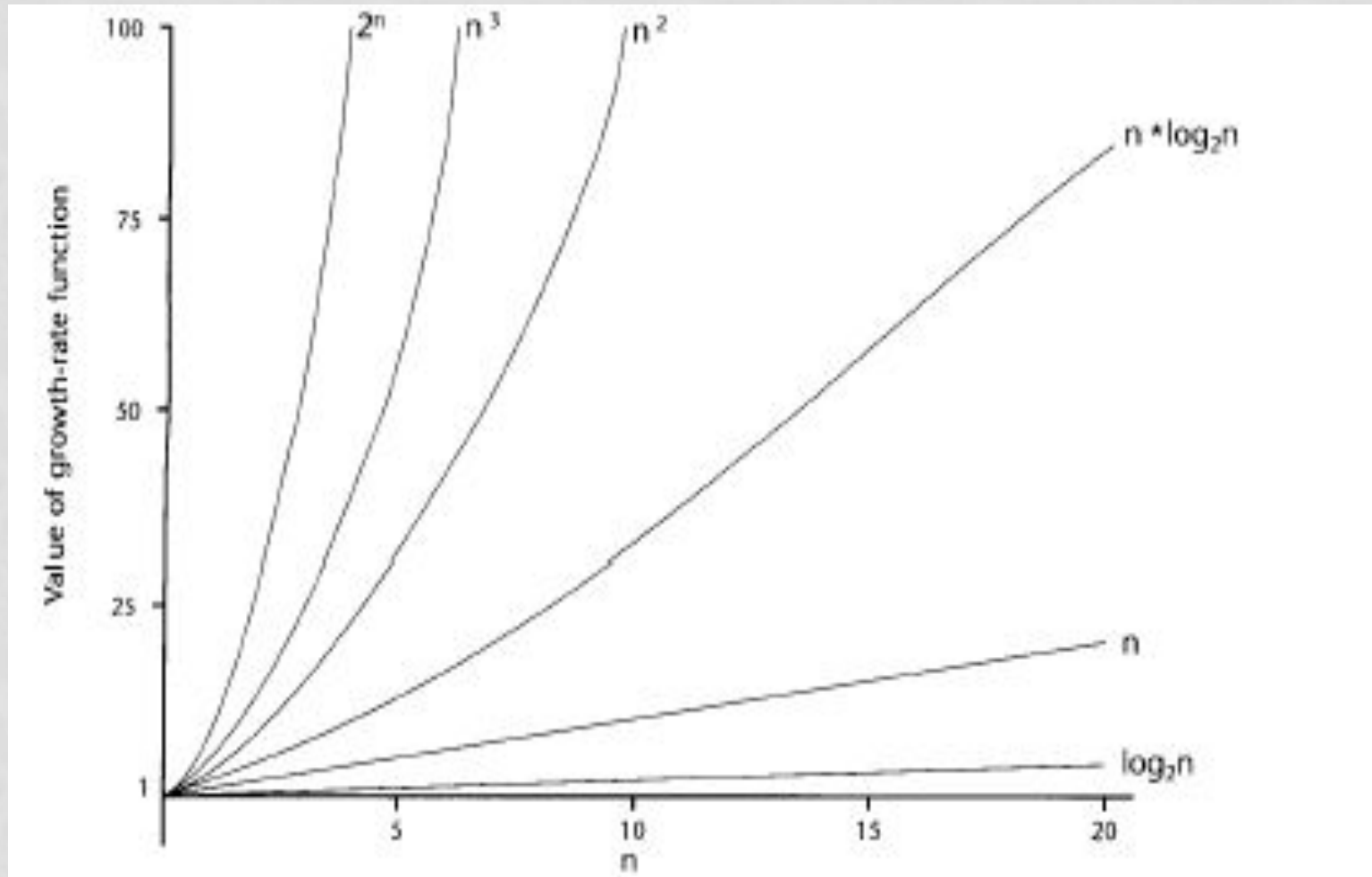
- All BST operations are $O(d)$, where d is tree depth
- minimum d is $d = \lfloor \log_2 N \rfloor$ for a binary tree with N nodes
 - What is the best case tree?
 - What is the worst case tree?
- So, best case running time of BST operations is $O(\log N)$

COMPLEXITY OF BST

For binary tree $b = 2$

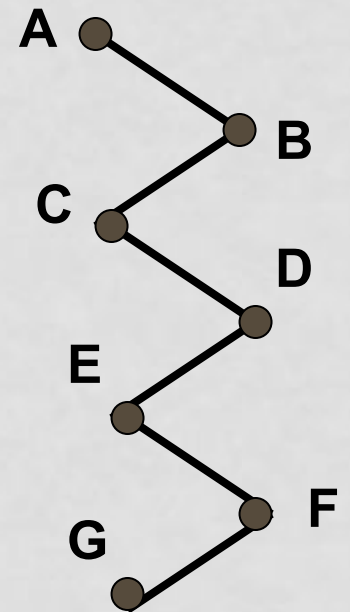
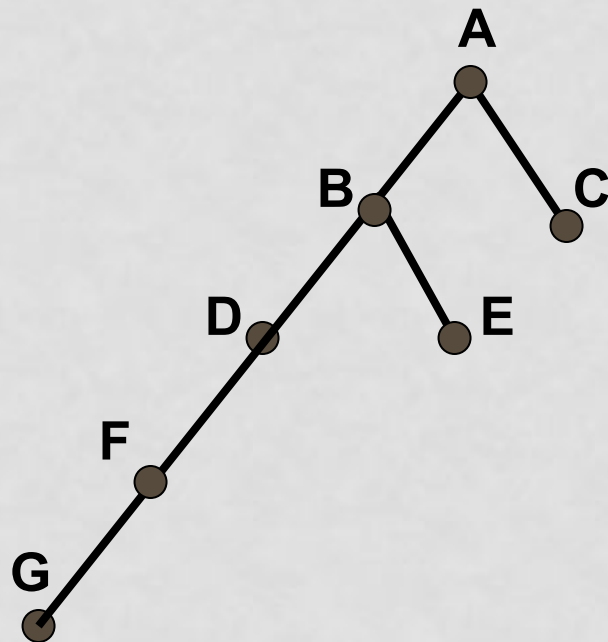
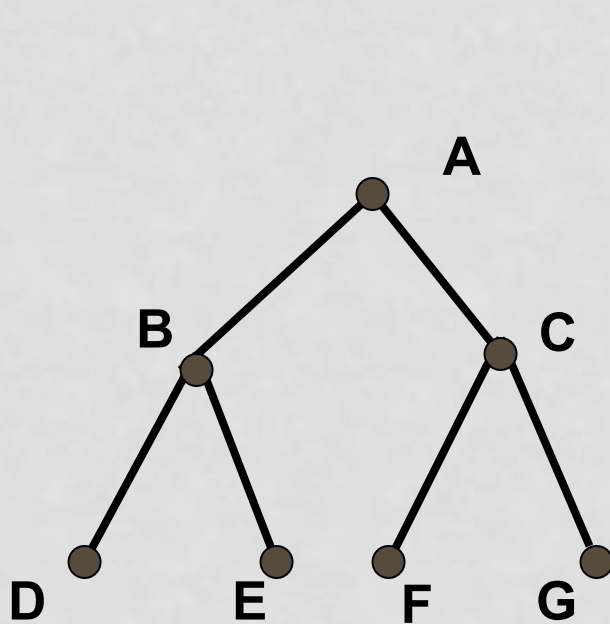


COMPARISON OF GROWTH RATE



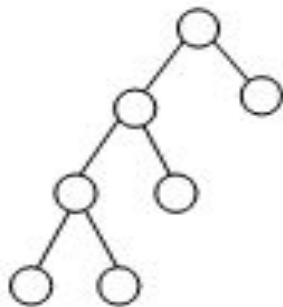
VARIATION IN SHAPES

- Trees come in Many Shapes.

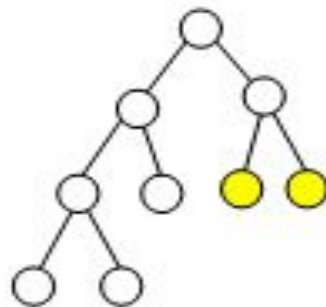


FULL, COMPLETE, AND PERFECT BINARY TREE

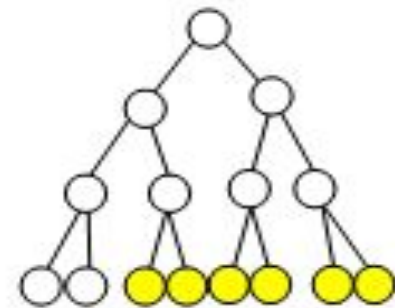
- If every node has either 0 or 2 children, a binary tree is called full.
- If the lowest $d-1$ levels of a binary tree of height d are filled and level d is partially filled from left to right, the tree is called complete.
- If all d levels of a height- d binary tree are filled, the tree is called perfect.



full



complete

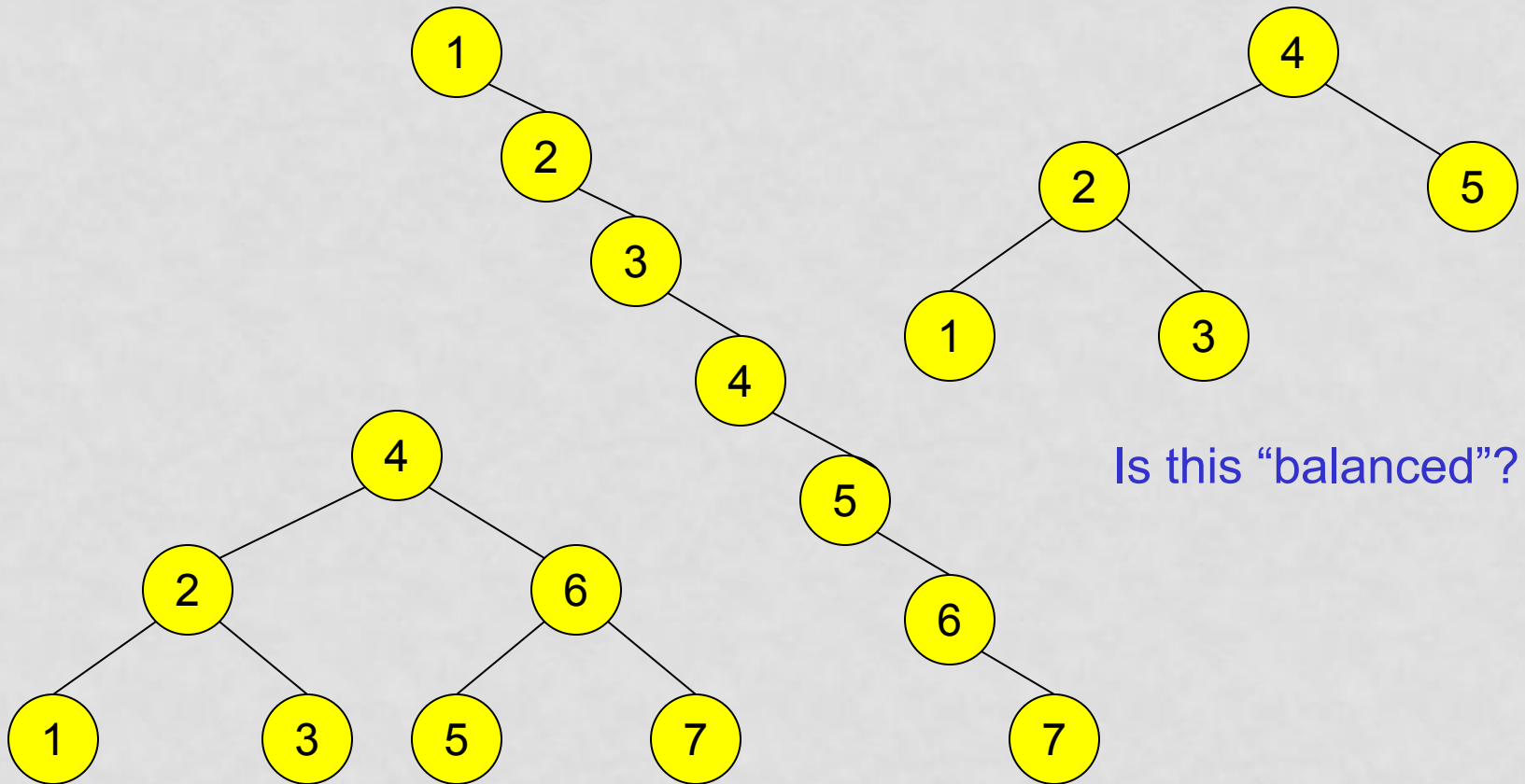


perfect

BINARY SEARCH TREE - WORST TIME

- Worst case running time is $O(N)$
 - What happens when you Insert elements in ascending order?
 - Insert: 2, 4, 6, 8, 10, 12 into an empty BST
 - Problem: Lack of “balance”:
 - compare depths of left and right subtree
 - Unbalanced degenerate tree

BALANCED AND UNBALANCED BST

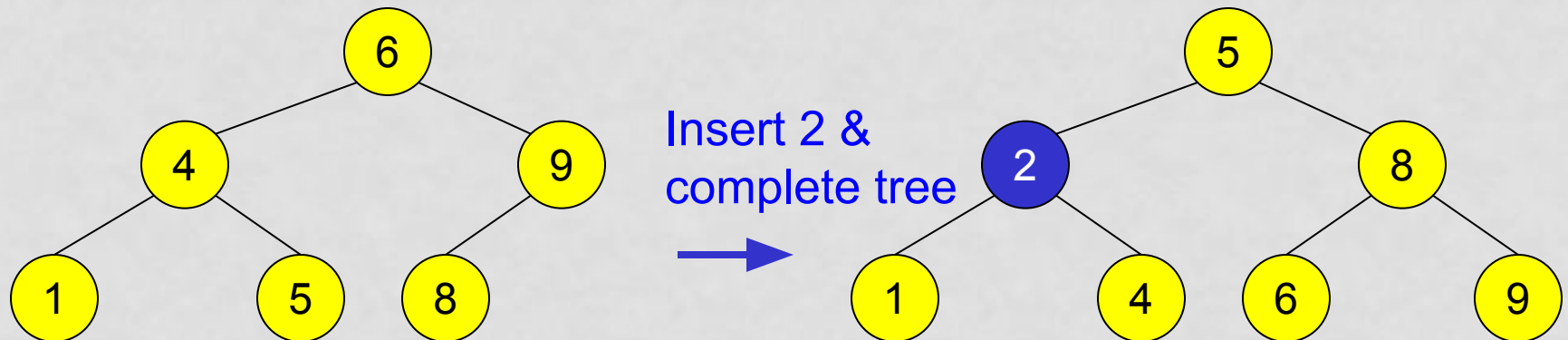


BALANCING BINARY SEARCH TREES

- Many algorithms exist for keeping binary search trees balanced
 - Adelson-Velskii and Landis ([AVL trees](#)) (height-balanced trees)
 - [Splay trees](#) and other self-adjusting trees
 - [B-trees](#) and other multiway search trees

PERFECT BALANCE

- Want a **complete tree** after every operation
 - tree is full except possibly in the lower right
- This is expensive
 - For example, insert 2 in the tree on the left and then rebuild as a complete tree



AVL - GOOD BUT NOT PERFECT BALANCE

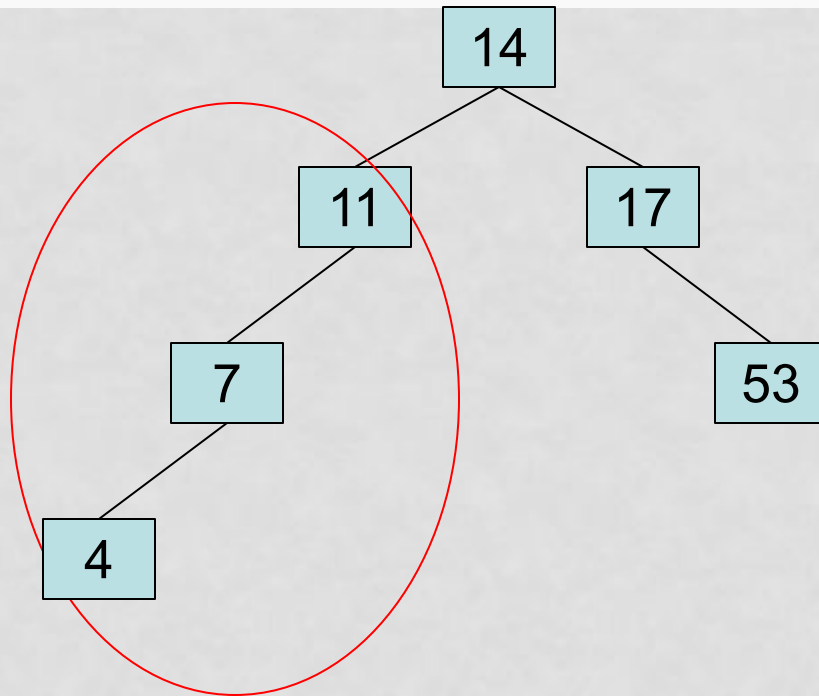
- AVL trees are height-balanced binary search trees
- Balance factor of a node
 - Balance Factor = height(left subtree) - height(right subtree)
- An AVL tree has balance factor calculated at every node
 - For every node, heights of left and right subtree can differ by no more than 1 i.e. $BF \in \{-1, 0, 1\}$
 - Store current heights in each node

ROTATIONS

- If $BF \notin \{-1, 0, 1\}$ i.e. $BF \in \{\pm 2\}$, then the node with $BF > 1$ undergoes rotation to balance the tree. Four types of rotation exists, depending upon the type of imbalance.
 - Left – Left
 - Right – Right
 - Left – Right
 - Right - Left

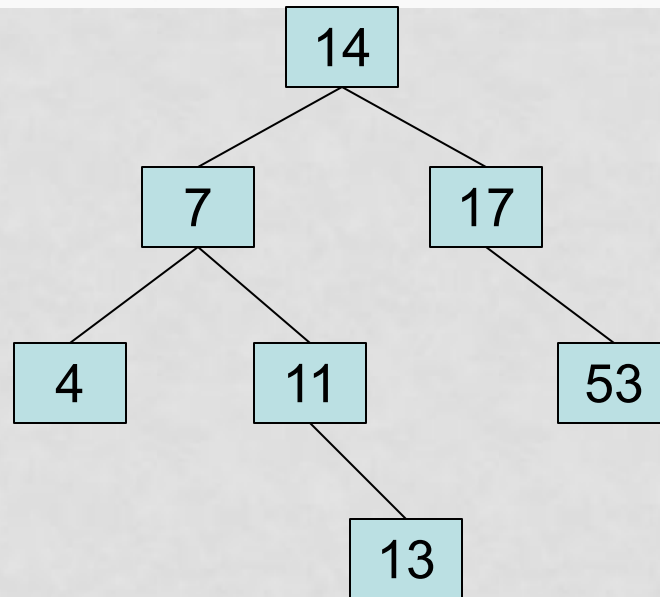
AVL Tree Example:

- Insert 14, 17, 11, 7, 53, 4, 13 into an empty AVL tree



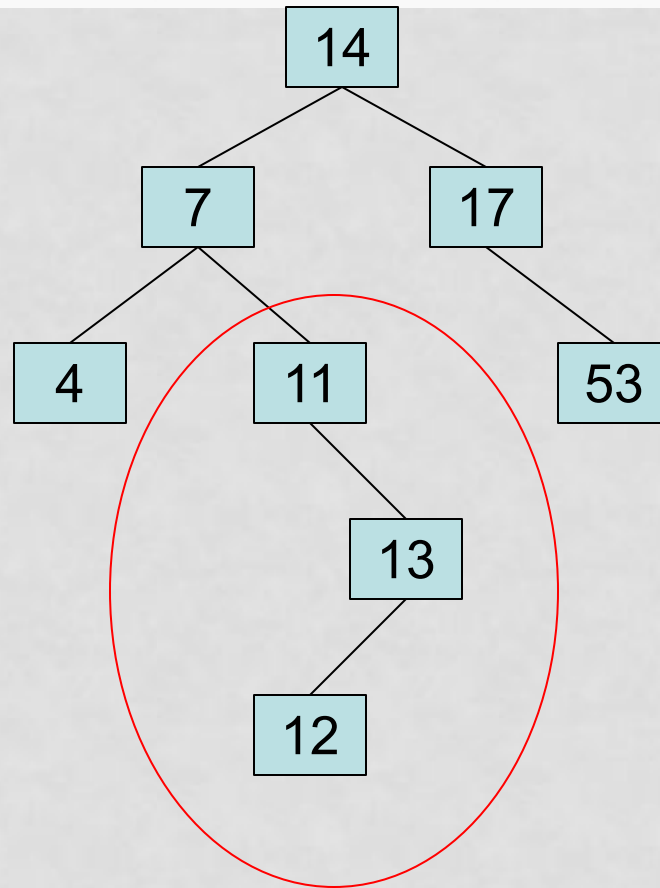
AVL Tree Example:

- Insert 14, 17, 11, 7, 53, 4, 13 into an empty AVL tree



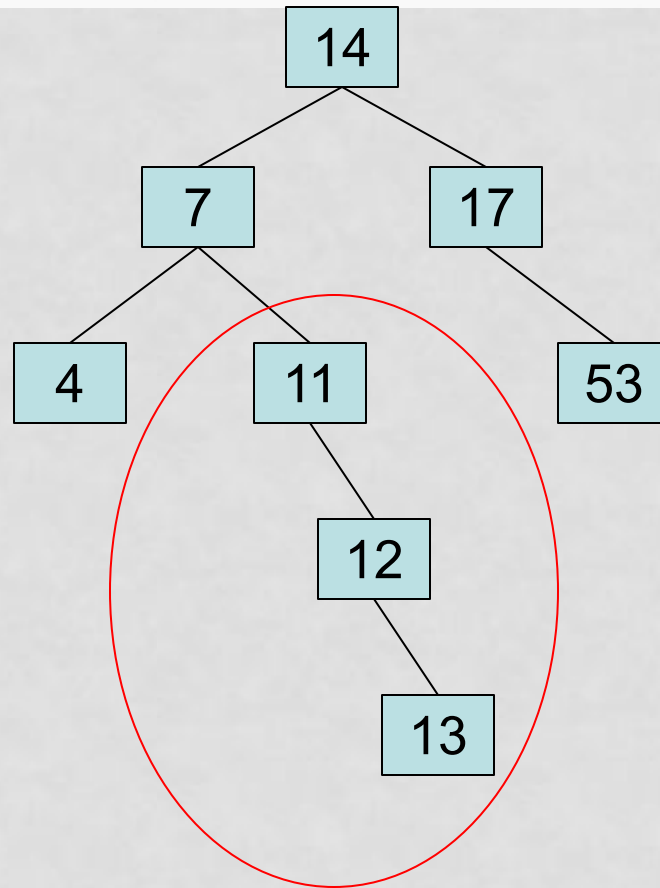
AVL Tree Example:

- Now insert 12



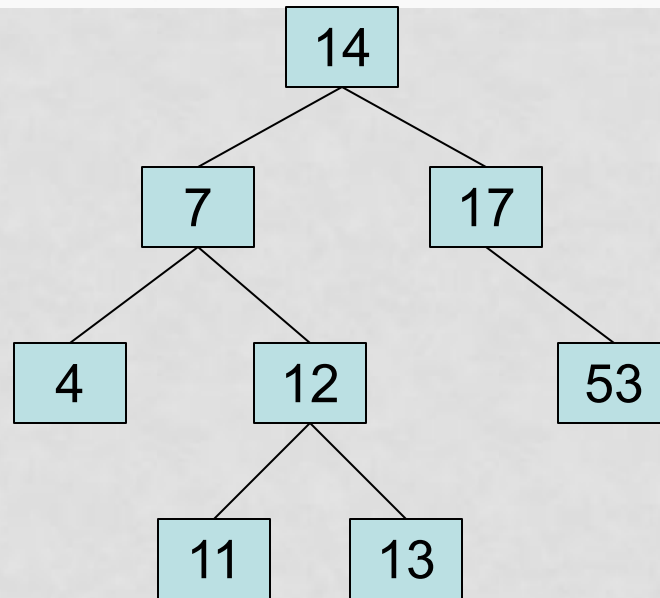
AVL Tree Example:

- Now insert 12



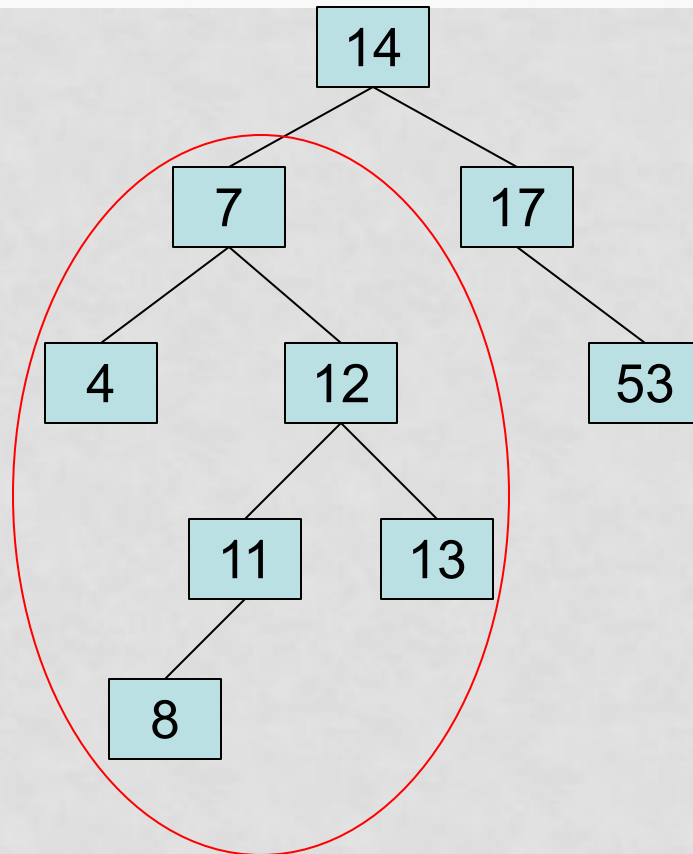
AVL Tree Example:

- Now the AVL tree is balanced.



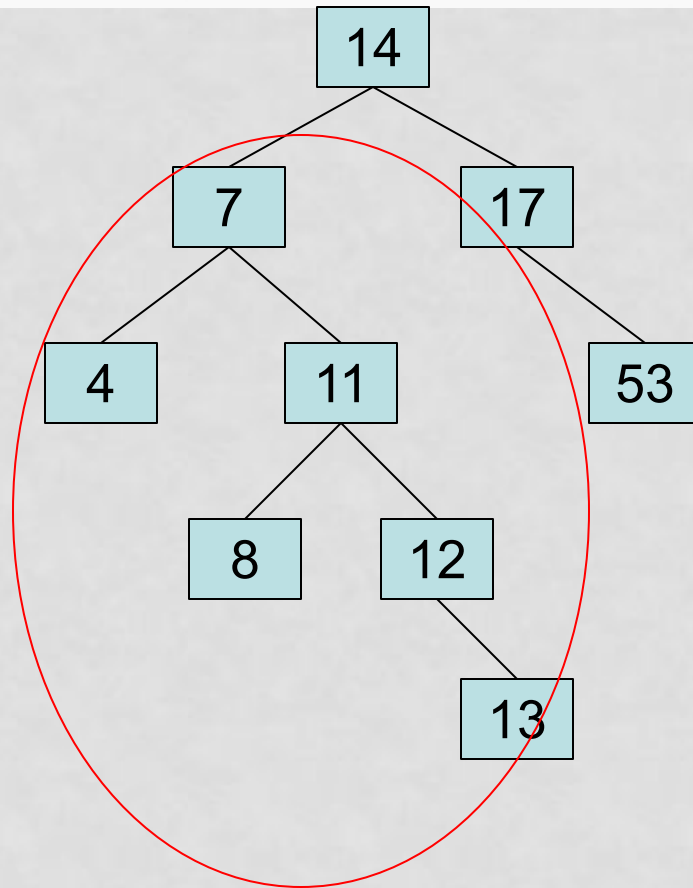
AVL Tree Example:

- Now insert 8



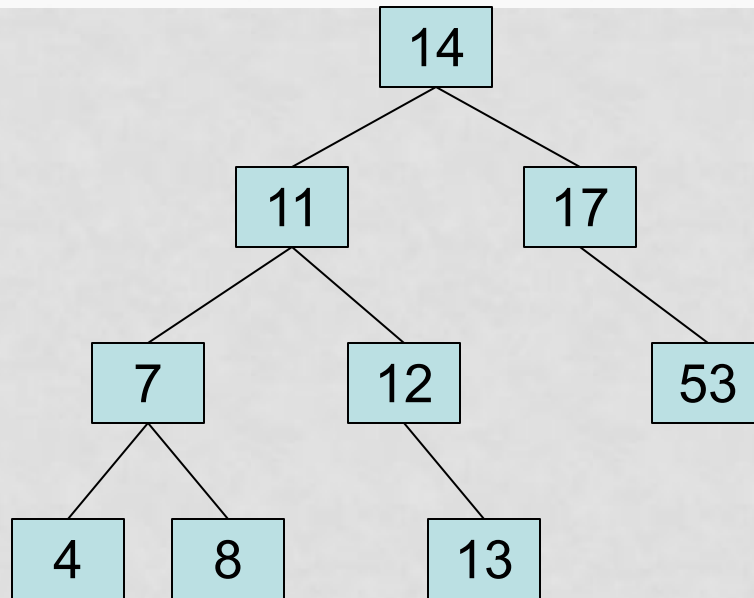
AVL Tree Example:

- Now insert 8



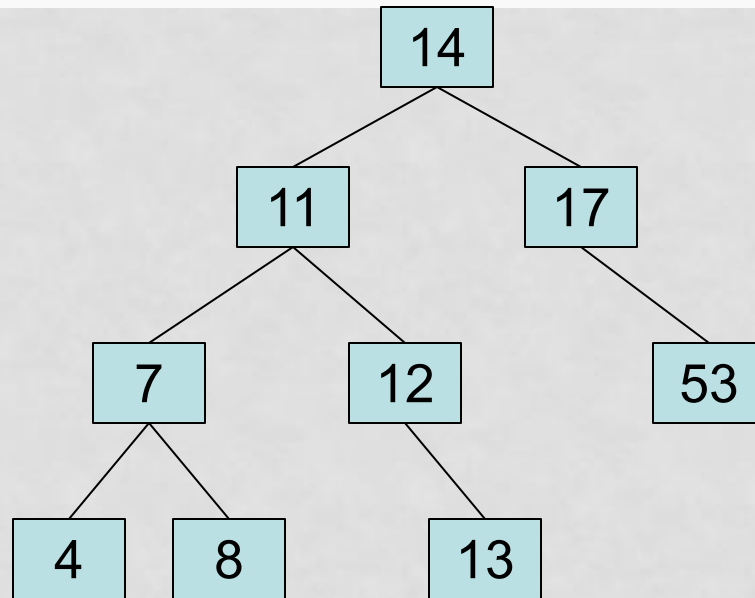
AVL Tree Example:

- Now the AVL tree is balanced.



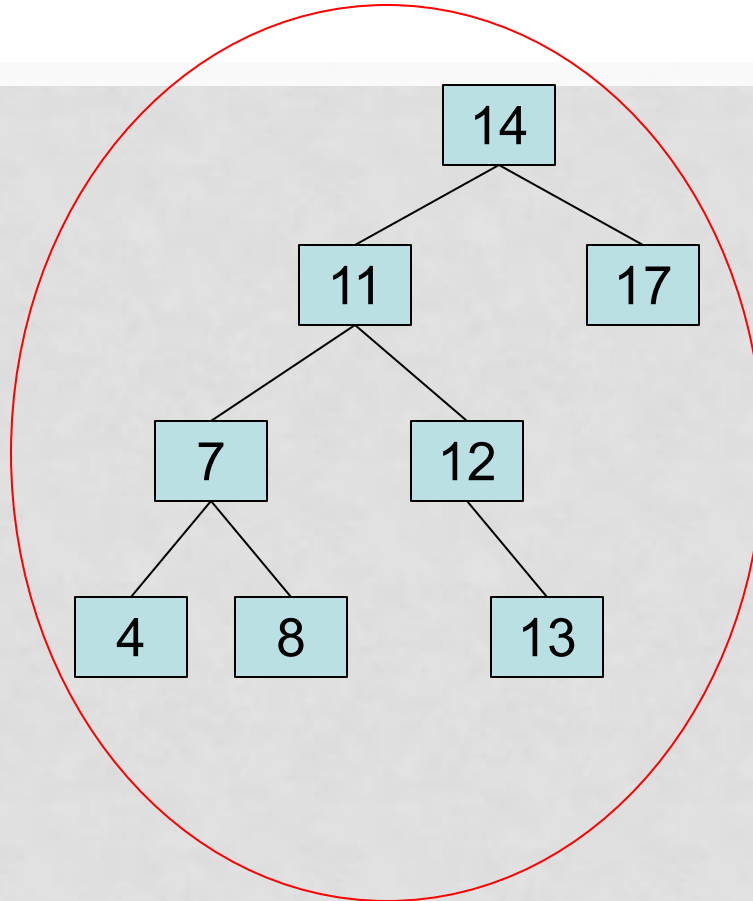
AVL Tree Example:

- Now remove 53



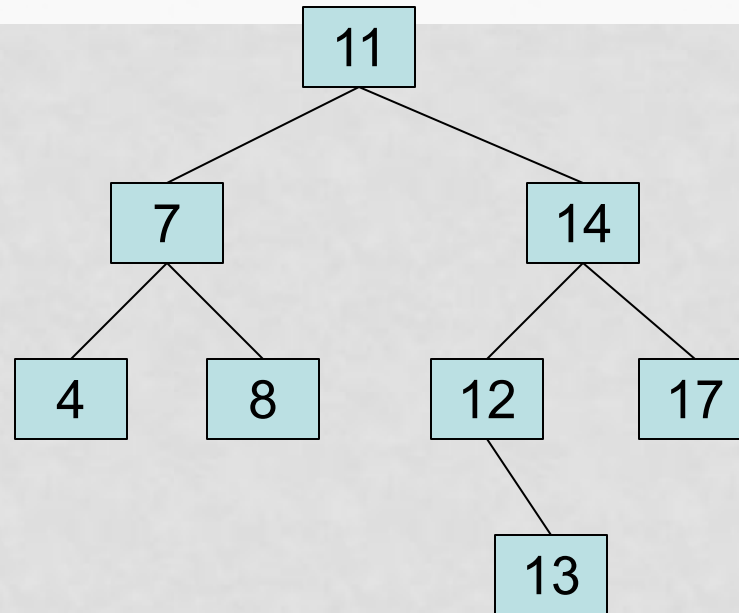
AVL Tree Example:

- Now remove 53, unbalanced



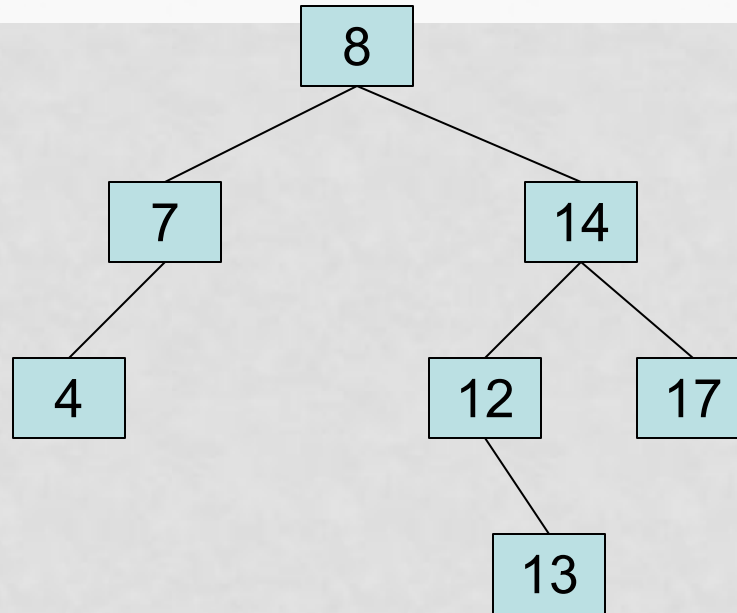
AVL Tree Example:

- **Balanced! Remove 11**



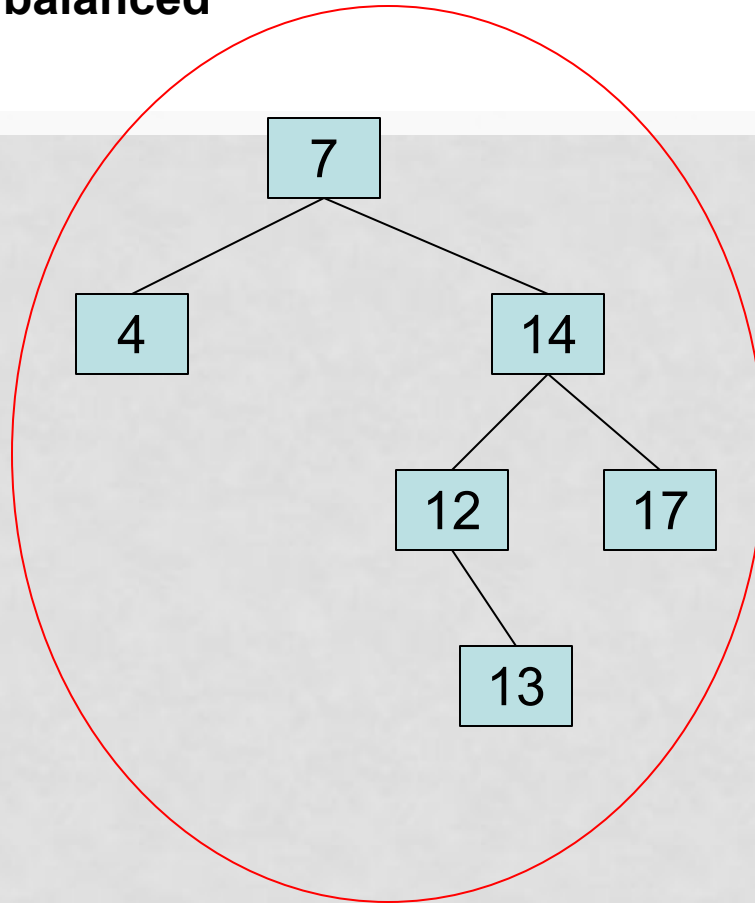
AVL Tree Example:

- Remove 11, replace it with the largest in its left branch



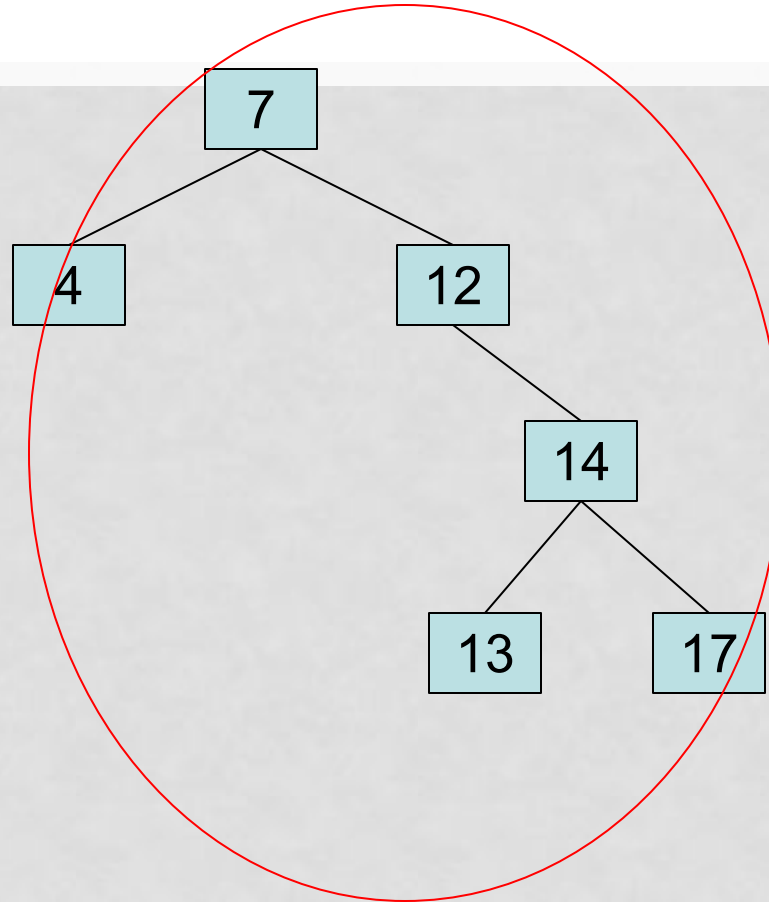
AVL Tree Example:

- Remove 8, unbalanced



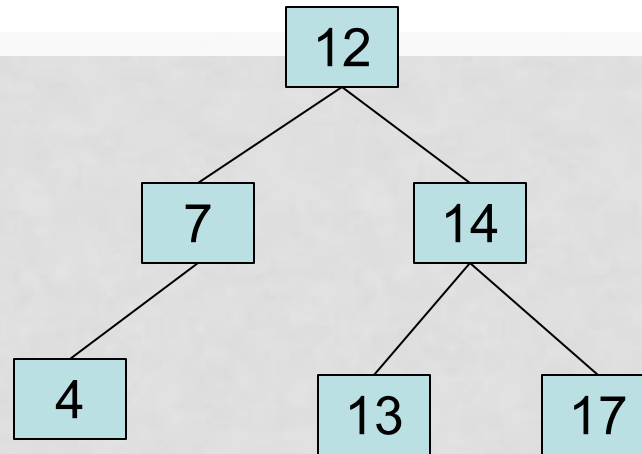
AVL Tree Example:

- Remove 8, unbalanced



AVL Tree Example:

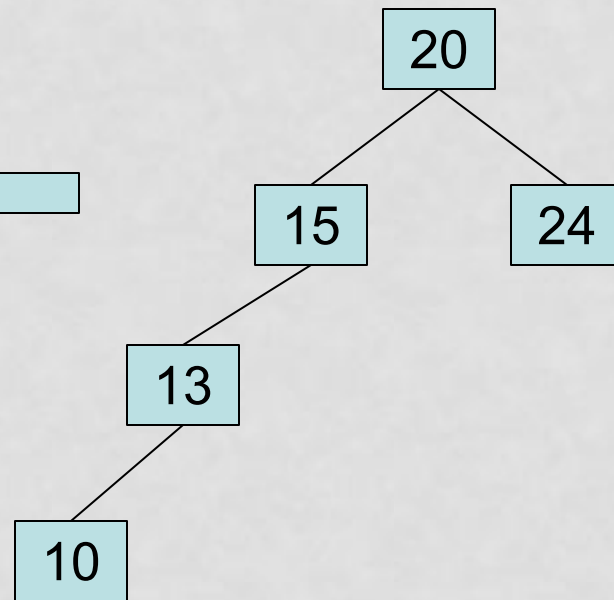
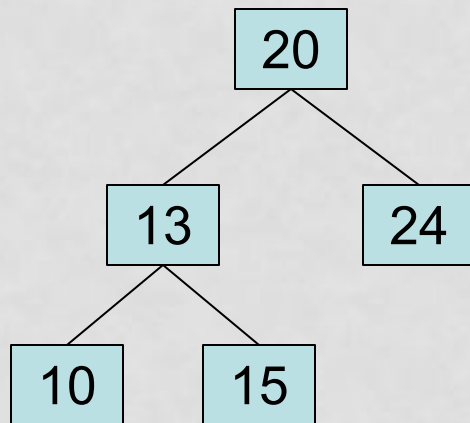
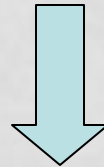
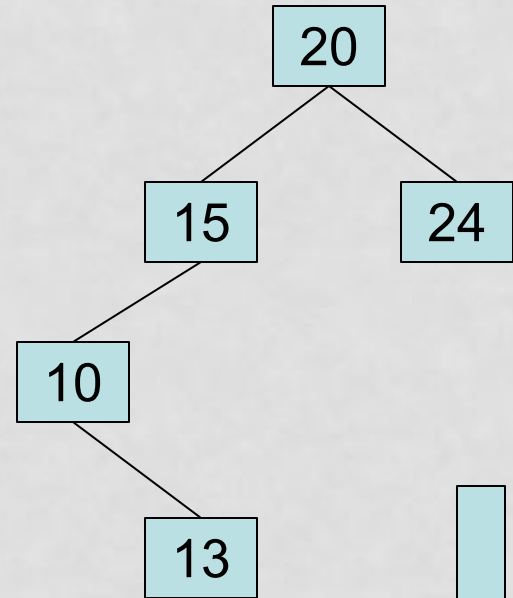
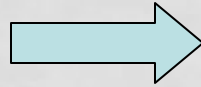
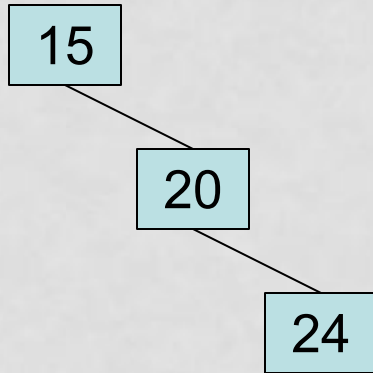
Balanced!!



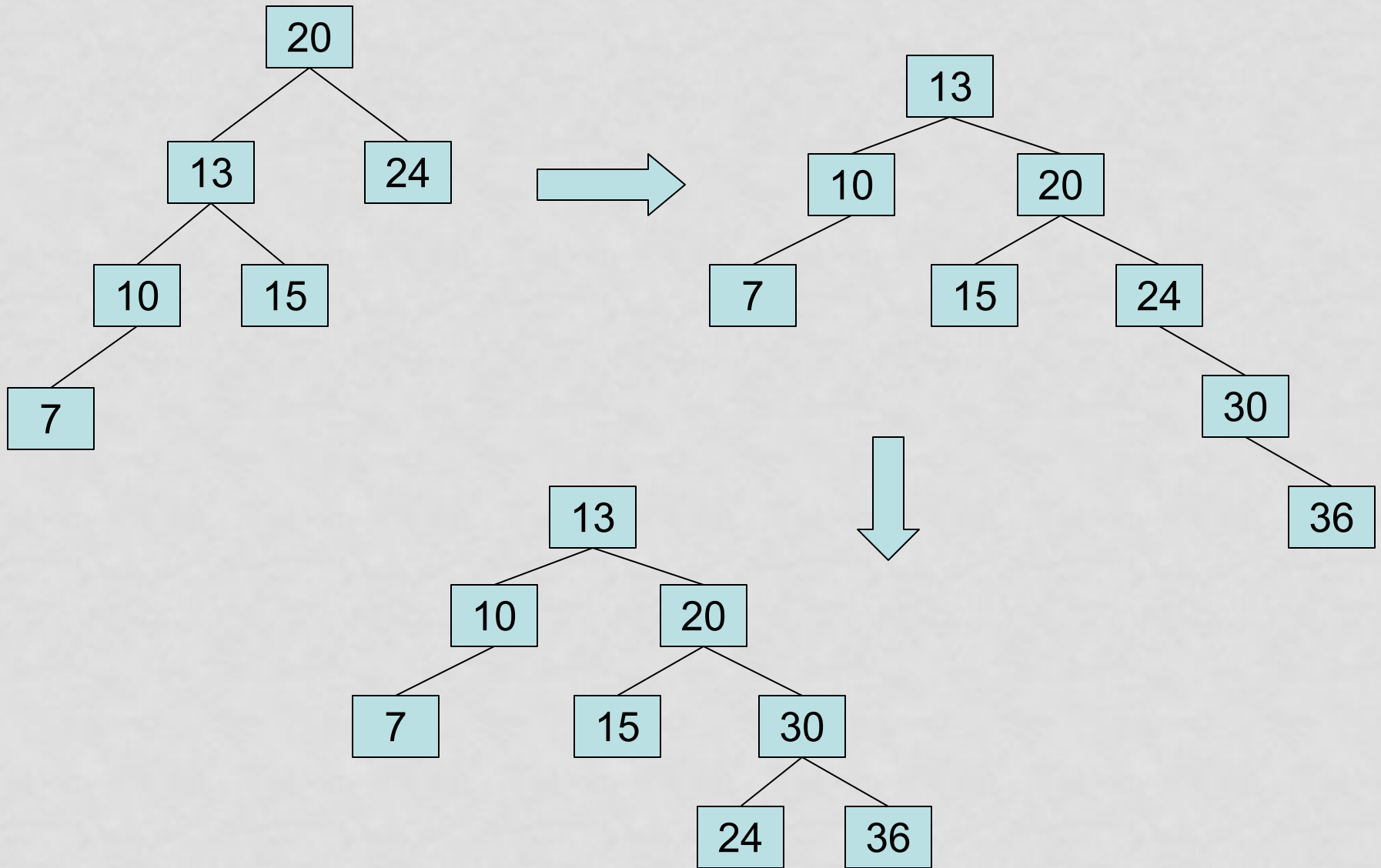
IN CLASS EXERCISES

- Build an AVL tree with the following values:
15, 20, 24, 10, 13, 7, 30, 36, 25
Delete 25 , 13, 24

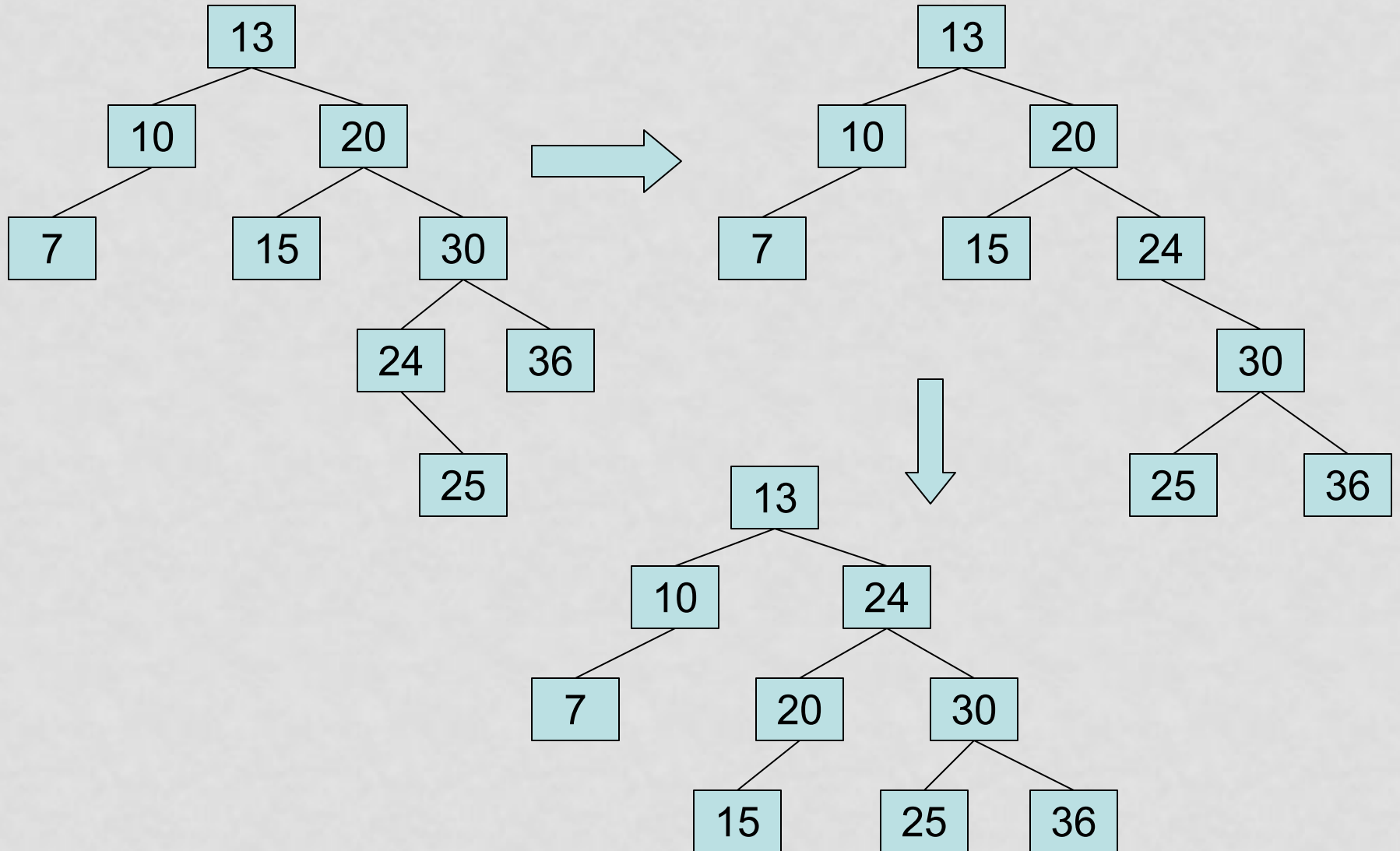
15, 20, 24, 10, 13, 7, 30, 36, 25



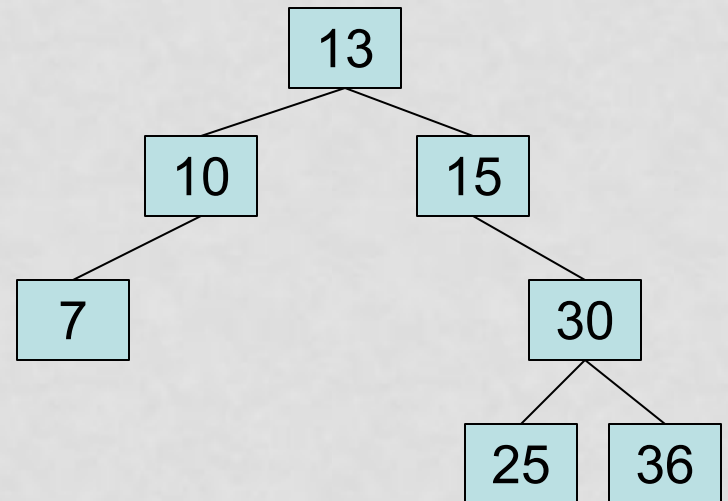
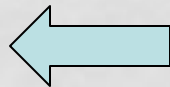
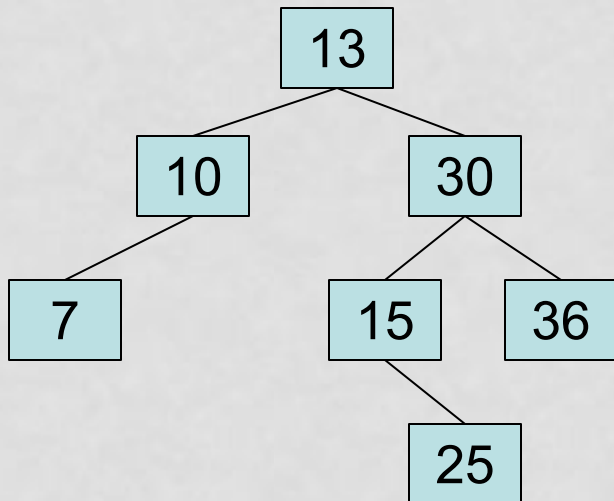
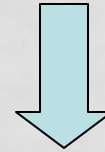
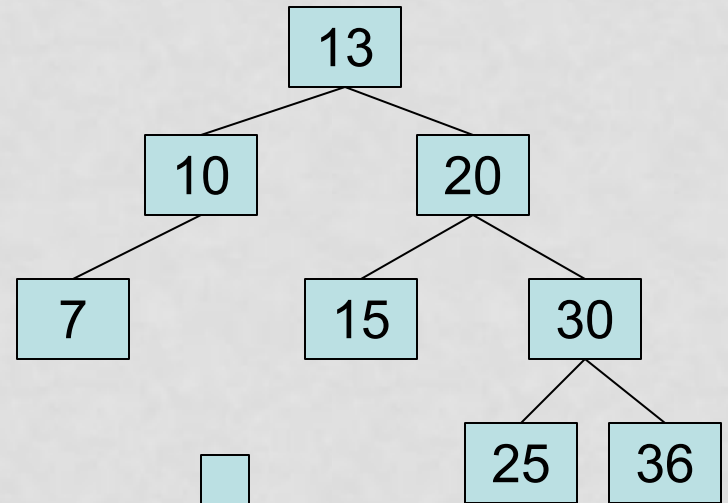
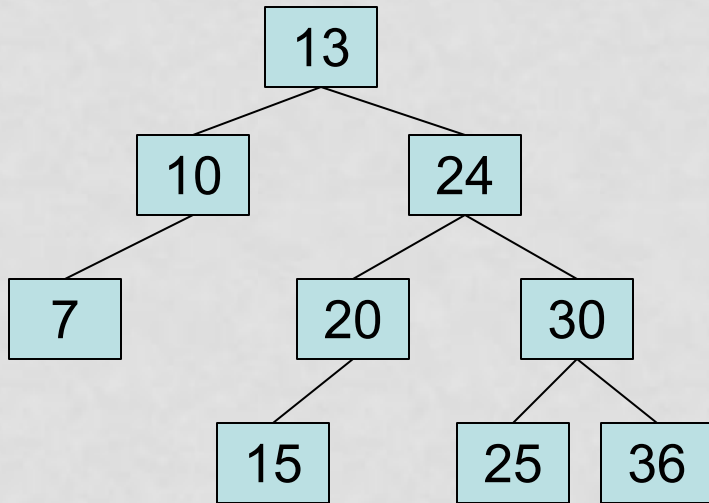
15, 20, 24, 10, 13, 7, 30, 36, 25



15, 20, 24, 10, 13, 7, 30, 36, 25



Remove 24 and 20 from the AVL tree.



ANY QUESTION?

- You can email me your queries regarding this lecture to course email id OR my personal email id.
- I will reply you through email or may entertain your queries in next online Lecture
 - datastructuresalgorithm@gmail.com
- You can ask your questions in live Q/A Session on Microsoft Teams
 - Syndicate A – Friday 1130-1230 hrs (Team code: **hws7wse**)
 - Syndicate B – Wednesday 1400 – 1500 hrs (Team code: **rsxxc9q**)

READING

- Chapter 12, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.

HELP

- <https://www.youtube.com/watch?v=up1FY5l8CEk>
- [https://www.youtube.com/watch?v= 8qqIVH5NC0](https://www.youtube.com/watch?v=8qqIVH5NC0)